



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

PROYECTO DE CARRERA: INGENIERÍA INFORMÁTICA

UNIDAD CURRICULAR : LENGUAJES Y COMPILADORES

SEMESTRE VII – 2026 – I

Análisis Léxico

Profesor:

Marquez, Felix

Estudiantes:

Frontado, Keiner C.I. 32.059.780

Martínez, Giovanni C.I. 31.782.337

Morales, Edgar C.I. 30.631.492.

Ortega, Mansour C.I. 28.785.297

Ciudad Guayana, julio 2026

Introducción

El desarrollo y la evolución de los procesadores de lenguaje constituyen uno de los pilares fundamentales de la ciencia de la computación y la ingeniería de software. Todo proceso de compilación e interpretación de un programa estructurado se fundamenta en una serie de fases secuenciales que transforman un texto fuente escrito en un lenguaje de alto nivel en instrucciones ejecutables por una máquina o en una representación intermedia estructurada. Dentro de esta arquitectura, el análisis léxico representa la primera etapa crítica del procesamiento, encargada de escanear la secuencia de caracteres del archivo fuente, identificar componentes léxicos válidos (tokens), descartar elementos irrelevantes y reportar anomalías sintácticas iniciales.

A fin de profundizar tanto en las bases teóricas como en las aplicaciones prácticas de la teoría de autómatas y lenguajes formales, el presente informe aborda el estudio, diseño e implementación de herramientas de análisis sintáctico y léxico orientadas a diferentes dominios tecnológicos:

Se examina la formulación matemática y teórica del autómata de pila como extensión de los autómatas finitos mediante la inclusión de una memoria auxiliar con disciplina LIFO. Se analiza su papel determinante en el reconocimiento de Lenguajes Libres de Contexto (LLC) y su pertinencia como modelo teórico para la fase de análisis sintáctico (parsing).

Se presenta la construcción de un implementado nativamente en Python 3 orientada al análisis de archivos de configuración de infraestructura. Dicho escáner emplea patrones de expresiones regulares compilados en un autómata finito unificado para clasificar directivas, banderas, cadenas e identificadores, incorporando un sistema de control de posición y tolerancia a errores léxicos.

Se documenta el uso de la herramienta metacompiladora Flex para la generación automática de un analizador léxico orientado a un subconjunto del lenguaje Rust. Asimismo, se incluye un manual de usuario detallando el flujo de generación e integración con el compilador C (GCC), complementado con un análisis comparativo crítico sobre las limitaciones de Flex frente al paradigma LangSec (Seguridad Basada en la Teoría de Lenguajes) y el uso de generadores deterministas modernos como PLEX en entornos de ciberseguridad e inspección de red.

A través de esta estructura, se busca consolidar una visión integral que abarca desde la rigurosidad teórica de la jerarquía de Chomsky hasta las metodologías contemporáneas de implementación y fortificación de analizadores léxicos en la ingeniería de software moderna.

Autómata de pila

Un autómata de pila (AP) es una máquina teórica de reconocimiento que extiende al autómata finito incorporándole una memoria auxiliar de acceso restringido llamada pila (stack), la cual opera bajo la disciplina LIFO. Gracias a esta memoria adicional, el autómata de pila puede reconocer lenguajes libres de contexto (LLC), es decir, lenguajes que pueden ser generados por una gramática de libre contexto (GLC) — el mismo tipo de gramática estudiado en el tema anterior para el analizador sintáctico.

Formalmente, un autómata de pila se define como una séptupla:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Componente	Descripción
Q	Conjunto finito de estados.
Σ	Alfabeto finito de símbolos de entrada.
Γ	Alfabeto finito de símbolos de pila (stack alphabet).
δ	Función de transición: $Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow$ subconjuntos finitos de $Q \times \Gamma^*$. Indica, dado el estado actual, el símbolo leído (o ϵ) y el símbolo en el tope de la pila, a qué estado se pasa y qué se apila o desapila.
$q_0 \in Q$	Estado inicial.
$Z_0 \in \Gamma$	Símbolo inicial de la pila (marca de fondo de pila).
$F \subseteq Q$	Conjunto de estados de aceptación.

A diferencia de la función de transición de un AFD (que solo depende del estado y el símbolo de entrada), la de un autómata de pila también depende del símbolo que está en el tope de la pila, y su resultado no es un único estado sino un conjunto de pares (estado, cadena a apilar), lo que evidencia que el autómata de pila es, por definición, un modelo no determinista en su forma más general.

En cada movimiento el autómata puede realizar una de tres operaciones sobre la pila:

- Push (apilar): coloca uno o más símbolos nuevos sobre el tope.
- Pop (desapilar): retira el símbolo del tope.
- Sin cambio: sustituye el tope por sí mismo (no se apila ni desapila).

La cadena de entrada es aceptada si, al consumirla por completo, el autómata se encuentra en un estado final (aceptación por estado final) o si la pila queda vacía (aceptación por pila vacía); ambos criterios son equivalentes en poder de reconocimiento, es decir, para todo AP que acepta por un criterio existe un AP equivalente que acepta por el otro.

Ejemplo: Lenguaje $L = \{ a^n b^n \mid n \geq 0 \}$

Este es el ejemplo que demuestra la necesidad de la pila: reconocer cadenas formadas por una secuencia de "a" seguida de la misma cantidad de "b". Este lenguaje no es regular (no puede reconocerse con un AFD ni un AFND), porque requiere "contar" una cantidad arbitraria de símbolos, algo que la memoria finita de estados no puede hacer para cualquier n.

Definición del autómata:

- $Q = \{ q_0, q_1, q_2 \}$
- $\Sigma = \{ a, b \}$
- $\Gamma = \{ a, Z_0 \}$
- Estado inicial: q_0 Símbolo inicial de pila: Z_0 $F = \{ q_2 \}$

Función de transición:

Transición	Acción	Significado
$\delta(q_0, a, Z_0) = (q_0, aZ_0)$	push a	Se lee la primera 'a' y se apila sobre el fondo.
$\delta(q_0, a, a) = (q_0, aa)$	push a	Cada nueva 'a' leída se apila.
$\delta(q_0, b, a) = (q_1, \epsilon)$	pop	La primera 'b' desapila una 'a' y cambia de fase.
$\delta(q_1, b, a) = (q_1, \epsilon)$	pop	Cada 'b' siguiente desapila una 'a'.

$\delta(q1, \epsilon, Z0) = (q2, Z0)$	sin cambio	Si solo queda Z0 (se agotaron las a's), se acepta.
---------------------------------------	------------	--

Traza de ejecución para la cadena de entrada "aaabbb":

Paso	Símbolo leído	Estado	Contenido de la pila (tope a la izquierda)
0	—	q0	Z0
1	a	q0	a Z0
2	a	q0	a a Z0
3	a	q0	a a a Z0
4	b	q1	a a Z0
5	b	q1	a Z0
6	b	q1	Z0
7	ϵ	q2 (acepta)	Z0

Obsérvese cómo la pila crece con cada 'a' y decrece exactamente en la misma proporción con cada 'b'; si la cadena tuviera una cantidad distinta de a's que de b's, al terminar la entrada la pila no contendría únicamente Z0 (o se intentaría desapilar sobre una pila sin el símbolo esperado), y la cadena sería rechazada.

Actividad 2: Analizador Léxico para Archivos Dockerfile

1. Descripción de la solución

Para el desarrollo de la Actividad 2, se construyó un analizador léxico desde cero en Python 3 orientado a la verificación léxica de archivos de configuración Dockerfile.

Dado que la sintaxis de Docker se basa en componentes regulares (instrucciones en mayúsculas, banderas opcionales, cadenas de texto y argumentos), la tokenización se implementó mediante el módulo nativo re, compilando un conjunto de expresiones regulares en un Autómata Finito Unificado con grupos nombrados ((?P<NOMBRE>patrón)).

2. Especificación Léxica y Componentes Reconocidos

El lexer reconoce y clasifica los siguientes componentes:

- **Directivas Principales (KW_*)**: Palabras reservadas del motor Docker (FROM, RUN, CMD, WORKDIR, COPY, EXPOSE, ENV, ENTRYPOINT, AS, entre otras).
- **Banderas Opcionales (FLAG)**: Modificadores de comandos (ej. --from=builder).
- **Cadenas de Texto (STRING)**: Argumentos delimitados por comillas simples o dobles ("...", '...').
- **Símbolos y Delimitadores**: Paréntesis cuadrados ([,]), comas (,), signos de igual (=) y dos puntos (:).
- **Identificadores y Rutas (IDENTIFIER)**: Nombres de imágenes, etiquetas, comandos y rutas del sistema (node:18-alpine, /app, ./).
- **Comentarios y Espacios (COMMENT / SKIP)**: Se ignoran comentarios iniciados con # y espacios/tabulaciones para no contaminar la salida.
- **Manejo de Errores Léxicos (MISMATCH)**: Captura cualquier carácter fuera del alfabeto permitido en el contexto (ej. @, \$, ~), reportando su número de línea y columna exacta sin interrumpir la lectura del resto del archivo.

3. Explicación Algorítmica

- Construcción del Patrón Regex Unificado*: Se definen los patrones en orden de prioridad. Las directivas exactas van primero para evitar que sean absorbidas por la regla general de identificadores.
- Escaneo de la Cadena mediante Autómata Unificado*: La función `re.finditer()` recorre el texto de entrada evaluando los lexemas. Por cada coincidencia, determina el grupo (`kind`) que activó la regla.
- Cálculo de Posición (Línea y Columna)*: Se mantiene un contador de saltos de línea (`NEWLINE`). La columna se calcula dinámicamente restando la posición de inicio de la coincidencia actual (`mo.start()`) del inicio de la línea activa (`line_start`).
- Tratamiento y Recuperación de Errores*: Si la coincidencia cae en `MISMATCH`, se incrementa el contador de errores, se despliega una alerta detallada en consola y la ejecución continúa desde el siguiente carácter.

4. Resultados de las Pruebas de Ejecución

Prueba 1: Configuración Básica de Aplicación Web (Dockerfile.1)

- *Objetivo*: Validar la tokenización de una imagen estándar de [Node.js](https://nodejs.org/).
- *Resultado*: Reconocimiento correcto de instrucciones directas (FROM, WORKDIR, COPY, RUN), delimitadores de arreglo JSON para el comando CMD y número de puerto en EXPOSE. Análisis finalizado sin errores.

Prueba 2: Construcción Multietapa / Multi-stage Build (Dockerfile.2)

- *Objetivo*: Verificar estructuras avanzadas como alias de construcción y banderas.
- *Resultado*: El lexer aisló correctamente la palabra clave AS, la etiqueta de la etapa previa (builder), la bandera --from=builder asociada al token FLAG y asignaciones de variables de entorno (ENV PORT=8080).

Prueba 3: Detección y Tolerancia a Errores Léxicos (Dockerfile.error)

- *Objetivo*: Evaluar la respuesta del sistema ante sintaxis no reconocida o caracteres inválidos.
- *Resultado*: El lexer detectó exitosamente 3 errores léxicos específicos:
 - Carácter '@' en la Línea 4, Columna 17.
 - Carácter '~' en la Línea 4, Columna 23.
 - Carácter '\$' en la Línea 5, Columna 8.El programa completó la tokenización de todo el documento reportando exactamente el resumen de errores encontrados.

5. Instrucciones de Ejecución

Ubicarse en el directorio del proyecto:

```
cd ~/lexer_docker
```

Ejecutar el script indicando el archivo a analizar:

```
python3 lexer_docker.py Dockerfile.1
python3 lexer_docker.py Dockerfile.2
python3 lexer_docker.py Dockerfile.error
```


Actividad 3: Lexer para un Subconjunto de Rust con Metacompilador

En esta actividad se aborda el diseño y la construcción de un analizador léxico (lexer) para un lenguaje L, definido como un subconjunto formal del lenguaje de programación Rust. Para este propósito, en lugar de codificar la lógica del reconocimiento desde cero, se emplea Flex (Fast Lexical Analyzer Generator), un metacompilador ampliamente utilizado en la ingeniería de lenguajes para automatizar la generación de escáneres léxicos en C.

El desarrollo de esta actividad abarca no solo la implementación técnica de las reglas en el archivo de especificación de Flex, sino también la documentación completa del entorno de compilación (mediante un manual de usuario) y la publicación del código fuente en un repositorio formal .

Manual de Usuario del Metacompilador Flex

1. Introducción y Propósito de Flex

Flex (Fast Lexical Analyzer Generator) es una herramienta de metacompilación diseñada para la generación automática de analizadores léxicos (lexers o scanners). Su función principal es transformar una especificación formal basada en reglas y expresiones regulares en un programa ejecutable escrito en lenguaje C.

En el desarrollo de compiladores, el analizador léxico constituye la primera fase del procesamiento, encargándose de leer el programa fuente carácter por carácter y agruparlos en componentes léxicos válidos (tokens) e identificar errores léxicos. Flex automatiza este proceso construyendo internamente un Autómata Finito Determinista (AFD) eficiente a partir de las expresiones regulares definidas por el usuario, evitando la necesidad de codificar la tabla de transiciones y el bucle de lectura de forma manual.

2. Estructura General de un Archivo de Especificación (.l)

Los archivos de entrada para Flex utilizan tradicionalmente la extensión .l. El archivo se divide en tres secciones principales delimitadas obligatoriamente por el símbolo %%:

```
%{  
  /* Sección 1: Definiciones y código C de cabecera */  
}%  
  
/* Opciones de Flex y Definiciones Léxicas */
```

```
%%  
/* Sección 2: Reglas y Acciones (Expresiones Regulares) */  
%%  
  
/* Sección 3: Código C de Usuario y Función Principal (main) */
```

Detalle de las Secciones:

- **Sección de Definiciones (Encabezado):**
 - Bloque `%{ ... %}`: Todo el código C dentro de este bloque es copiado textualmente al inicio del archivo generado (`lex.yy.c`). Se utiliza principalmente para incluir librerías (`#include <stdio.h>`), definir constantes y declarar variables globales.
 - Directivas de opciones: Por ejemplo, `%option noyywrap`, la cual indica a Flex que no debe procesar múltiples archivos consecutivos al llegar al fin de lectura (EOF).
- **Sección de Reglas y Acciones:**
 - En esta sección se listan los patrones léxicos mediante Expresiones Regulares (Regex) seguidos de su correspondiente acción en bloque C `{ ... }`.
 - Nota importante: Los comentarios dentro de esta sección deben llevar sangría (espacio o tabulación) para no ser interpretados por Flex como patrones regex de la columna cero.
- **Sección de Código de Usuario:**
 - Contiene funciones C complementarias y obligatoriamente la función `main()` para ejecutar el análisis. Aquí se gestiona la apertura del archivo de entrada y la llamada al motor de análisis.

3. Variables y Funciones Clave de Flex

Flex provee variables y funciones globales integradas para facilitar la manipulación de los datos durante la tokenización:

- **yylex():** Función principal generada por Flex que inicia la lectura e interpretación del flujo de caracteres.

- **yytext:** Puntero a cadena de caracteres (char*) que almacena el lexema exacto que coincidió con la regla activa en ese momento.
- **yyin:** Puntero de archivo (FILE*) que indica a Flex la fuente de donde debe leer los caracteres. Por defecto es stdin, pero puede reasignarse a un archivo abierto con fopen().

4. Flujo de Compilación y Ejecución

El proceso para transformar una especificación léxica redactada en Flex hasta obtener el resultado final del análisis sobre un código fuente consta de tres etapas consecutivas e interconectadas:

Etapla 1: Generación del código fuente en C

- **Entrada:** El archivo de especificación léxica con extensión .l.
- **Proceso:** Se ejecuta el metacompilador Flex mediante el comando flex (o win_flex.exe en sistemas Windows). Flex procesa el archivo de entrada, analiza las expresiones regulares definidas en las reglas y genera automáticamente la tabla de transiciones de un Autómata Finito Determinista (AFD).
- **Salida:** Como resultado de este proceso, Flex crea un archivo con código fuente en C llamado convencionalmente lex.yy.c, el cual contiene la función yylex() y toda la lógica necesaria para llevar a cabo el escaneo.

Etapla 2: Compilación del archivo generado

- **Entrada:** El archivo de código fuente en C recién generado (lex.yy.c).
- **Proceso:** Se utiliza un compilador de C estándar, como GCC, ejecutando el comando gcc lex.yy.c -o analizador_lexico. GCC traduce la lógica C a instrucciones de lenguaje de máquina y vincula las librerías necesarias para el manejo de archivos de entrada/salida estándar.
- **Salida:** Se obtiene un archivo binario ejecutable (por ejemplo, analizador_lexico en Linux/macOS o analizador_lexico.exe en Windows).

Etapla 3: Ejecución del Analizador Léxico

- **Entrada:** El binario ejecutable obtenido y el archivo de código fuente que se desea analizar (por ejemplo, prueba.rs).

- **Proceso:** Se ejecuta el binario desde la terminal pasándole como parámetro el archivo fuente (ejecutando `./analizador_lexico prueba.rs`). El programa abre el archivo, asigna su puntero a la variable `yyin` y comienza la lectura de la cadena de texto carácter por carácter.
- **Salida:** A medida que el autómata reconoce los patrones definidos, ejecuta las acciones programadas para cada regla, imprimiendo en pantalla la secuencia de tokens hallados con sus respectivos lexemas, o notificando en caso de encontrar un error léxico.

Comandos de Consola:

4.1 Generación del código fuente C:

Bash

```
flex archivo.l
```

En sistemas Windows se puede emplear: `win_flex.exe archivo.l` descargando un port de Flex y GNU Bison llamado WinFlexBison de GitHub, generando así el archivo `lex.yy.c`

4.2 Compilación del ejecutable:

Bash

```
gcc lex.yy.c -o analizador_lexico
```

Esto genera el binario ejecutable `analizador_lexico` o `analizador_lexico.exe` en Windows.

4.3 Ejecución sobre un archivo fuente:

```
./analizador_lexico programa_prueba.rs
```

Con el binario ejecutable ya generado, por medio del analizador léxico creado se puede ejecutar el archivo fuente y obtener las instrucciones definidas

5. Ventajas y Comparación con la Implementación Manual

- **Rapidez de Desarrollo:** Permite construir analizadores léxicos complejos en poco tiempo escribiendo especificaciones concisas basadas en expresiones regulares.
- **Mantenibilidad:** Modificar o añadir nuevos tokens requiere únicamente agregar una línea de regla en el archivo .l, sin necesidad de alterar la lógica del autómata.
- **Consideración:** Aunque facilita la creación rápida, puede resultar menos flexible en escenarios altamente personalizados o complejos que requieran un manejo de memoria y estados no estándares comparado con una implementación nativa desde cero.

Lenguajes en el área de seguridad informática donde aplicar FLEX

El diseño de analizadores léxicos para infraestructuras de ciberseguridad representa un desafío crítico frente a la proliferación de vulnerabilidades sistémicas de bajo nivel. El paradigma LangSec (Seguridad Basada en la Teoría de Lenguajes) advierte que delegar la validación sintáctica a motores de ejecución semántica fomenta la creación de "máquinas extrañas". Estos sub-autómatas no intencionados otorgan a los atacantes capacidades computacionales imprevistas mediante diferenciales de análisis, un riesgo endémico en tecnologías que históricamente han dependido de la herramienta generadora FLEX y el parser Bison para modelar sus Autómatas Finitos Deterministas. Para erradicar esta superficie de ataque, el diseño óptimo de políticas de seguridad exige trascender la sintaxis heredada hacia el ámbito de validadores formales y deterministas.

Limitaciones de FLEX y la Integración Teórica de PLEX

Tecnologías vanguardistas como YARA y Zeek ilustran las severas limitaciones operativas de FLEX en entornos de análisis crítico. YARA divide su procesamiento en componentes modulares (como `lexer.l` y `re_lexer.l`) para evitar un AFD monolítico, pero se ve forzado a reescribir funciones nativas de C y aplicar técnicas de retroceso de bajo nivel (`setjmp` y `longjmp`) para prevenir que las aserciones de punteros nulos colapsen la pila léxica. Por su parte, Zeek, al monitorizar tráfico a velocidades gigabit, requiere instanciar objetos C++ directamente desde el lexer para computar máscaras de subred, enfrentándose históricamente al truncamiento silente de cadenas (strings) mediado por inyecciones de bytes nulos. La aplicación teórica de PLEX en estos ecosistemas sustituiría el frágil manejo de excepciones por validadores léxicos estrictos basados en el principio de maximal munch, donde cada expresión regular se asocia a funciones de acción aisladas, garantizando una gestión de estado

intrínsecamente segura y sin dependencias cruzadas.

De manera análoga, el control de acceso obligatorio (MAC) en SELinux y el filtrado transaccional en ModSecurity exponen la rigidez del binomio FLEX/Bison. En SELinux, la integración de topologías de hardware modernas que utilizan caracteres imprevistos (como los dos puntos en descriptores de Android) provocó que su escáner `policy_scan.l` careciera de transiciones válidas, generando errores duros que detenían irreversiblemente el flujo de compilación. ModSecurity sufre asimetrías similares al extraer variables diferidas desde flujos HTTP heterogéneos, produciendo fallos silenciosos ante límites de memoria excedidos. En contraste, la aplicación de PLEX permite un manejo declarativo de excepciones de análisis; mediante decoradores en Python (ej. `@lexer.on_error`) o implementaciones tipadas en Rust, PLEX evita el colapso del Autómata Finito y permite una interrupción controlada y determinista del análisis sin sacrificar la expresividad sintáctica.

PLEX emerge como un generador léxico avanzado que abstrae la complejidad ofuscada de los AFD tradicionales. A diferencia de los motores basados en C puro, PLEX compila todos los patrones de expresiones regulares en un único AFD capaz de procesar el flujo en un tiempo directamente proporcional a la longitud de los caracteres escaneados, eliminando las ambigüedades de estado. Esto permite que las arquitecturas de seguridad restrinjan su lenguaje de entrada a los límites teóricos del modelo LangSec, asegurando que los datos malformados jamás alcancen los estratos de ejecución abstracta.

Característica Arquitectónica	Implementación Tradicional (FLEX / C)	Aplicación Moderna (PLEX / Rust o Python)	Implicación LangSec
Manejo de Errores	Excepciones fatales (YY_FATAL_ERROR), desbordamientos de búfer.	Decoradores tipados (on_error) y sistemas de pánico seguro (unwrap).	Mitiga bloqueos imprevisibles y desmantelamiento corrupto de la pila léxica.
Resolución de Estado	Sub-autómatas acoplados y	Evaluación paralela de expresiones	Erradica la generación de

	manipulaciones manuales de punteros.	regulares mediante <i>maximal munch</i> .	"máquinas extrañas" por asimetrías de análisis.
Integración de Tipos	Conversiones crudas y truncamiento silente por bytes nulos (\0).	Tipado fuerte con abstracción del tiempo de vida de la memoria (lifetimes).	Previene inyecciones estructurales y supresión semántica en flujos gigabit.

Como conclusión, el candidato idóneo para la aplicación de PLEX es el lenguaje de filtrado de libpcap, la biblioteca fundacional de captura de paquetes subyacente en sensores de red como tcpdump y sistemas de detección de intrusos. Actualmente, libpcap emplea scanner.l (FLEX) y grammar.y (Bison) para interpretar las reglas del filtro de paquetes Berkeley (BPF). Este enfoque heredado adolece de conflictos algorítmicos (shift/reduce conflicts) y es históricamente susceptible a fallos de segmentación cuando se exponen árboles de análisis desbordados ante entradas hostiles. La migración de este analizador hacia un entorno orquestado por PLEX disociaría completamente la validación léxica de los riesgos de corrupción de memoria inherentes a C, alienando la tecnología de filtrado con los axiomas de seguridad computacional.

La tokenización del lenguaje BPF utilizando PLEX redefiniría la robustez del escáner mediante primitivas inmutables y explícitamente delimitadas. En una implementación hipotética utilizando la macro `lexer!` de Rust, la extracción de direcciones de red complejas y máscaras de subred (CIDR) se declararía formalmente como `r"[0-9]{1,3}(\.[0-9]{1,3}){3}(/[0-9]{1,2})?" => Token::IPv4Literal(tok.parse().unwrap())`. Este modelo de tokenización transfiere la responsabilidad de la asignación de memoria desde el lexer hacia el compilador seguro de Rust. Del mismo modo, para identificadores del protocolo de capa de transporte, una regla estricta como `"tcp" | "udp" | "icmp" => Token::ProtocolIdent(tok)` aseguraría una categorización de terminales rígida, imposibilitando que las variaciones semánticas subviertan la inspección.

Finalmente, para la captura de operadores lógicos o delimitadores físicos que comúnmente constituyen el tejido de una "máquina extraña" en entornos heredados, PLEX forzaría el análisis perimetral absoluto. Operadores condicionales como "and", "or" y "not" retornarían constantes unitarias predefinidas. Cualquier secuencia cruda dentro de la trama de red que no coincida matemáticamente con la gramática especificada del BPF activaría el pánico tipado del sistema (por ejemplo, mediante la directiva `. mapeada a panic!("unexpected character")` o invocando un decorador `@lexer.on_error`), purgando el paquete defectuoso instantáneamente sin intentar una recuperación heurística. Esta rigurosa arquitectura de tokenización garantiza que el autómata finito actúe como una barrera teórica determinista, transformando la inestable superficie de ataque de la red en un dominio axiomáticamente seguro e inviolable.

Conclusión

El autómata de pila es la extensión natural del autómata finito que resulta de dotarlo de una memoria auxiliar tipo pila. Esta única adición cualitativa (memoria no acotada con disciplina LIFO) es suficiente para elevar su poder de reconocimiento de los lenguajes regulares a los lenguajes libres de contexto, permitiéndole resolver problemas de anidamiento y conteo que están fuera del alcance de cualquier AFD o AFND, sin importar cuántos estados se le agreguen.

Su utilidad práctica más relevante para esta asignatura es servir de fundamento teórico al analizador sintáctico de un compilador, de la misma manera en que el autómata finito fundamenta al analizador léxico: así como el lexer valida que los componentes léxicos de un programa sean válidos según un lenguaje regular, el parser (modelado conceptualmente como un autómata de pila) valida que esos componentes estén organizados según la gramática de libre contexto del lenguaje, verificando el balance y anidamiento correcto de sus estructuras.

Finalmente, la distinción entre autómata de pila determinista y no determinista (a diferencia de lo que ocurre entre AFD y AFND, donde ambos son equivalentes) tiene consecuencias prácticas directas: motiva por qué los lenguajes de programación reales se diseñan con gramáticas analizables deterministamente (LL/LR), de modo que puedan compilarse de forma eficiente.

Referencias bibliográficas

- Alonso, L. (s.f.). *Procesadores de lenguaje: Compiladores*. Universidad de Salamanca.
<http://avellano.fis.usal.es/~lalonso/procesadores/compiladores.pdf>
- Maruri Parada, E. Y. (2024). *Compendio sobre Flex y Bison*. Instituto Politécnico Nacional.
<https://es.scribd.com/document/785727315/flex-y-bison>
- Red Iberoamericana de Academias de Investigación. (2020). *Vanguardia tecnológica, industria 4.0 y sistemas inteligentes*. REDIBAI.
<https://redibai-myd.org/portal/wp-content/uploads/2020/01/8617-41-8.pdf>